

METHODS OF THREAD CLASS :

21 February 2025 17:07

- A **Daemon Thread** in Java is a low-priority thread that runs in the background to support other non-daemon (user) threads. The JVM automatically terminates daemon threads when all user threads finish execution.

First thing to remember when I create a simple program.

```
package com.st;
public class Buffer {
    public static void main(String[] args) {
        System.out.println("this is me");
    }
}
```

- So in this case one thread is created by JVM.
- That is the main thread.
- If you want to get its thread name then

```
System.out.println(Thread.currentThread().getName());
```

- If you want to change its thread name then you have to write

```
Thread.currentThread().setName("jatin");
System.out.println(Thread.currentThread().getName());
System.out.println(100/0);
```

➤ Daemon Thread

- Which run in the background of another thread
- It provide service to the thread.
- Eg : garbage collection , spell checker etc

Real-Life Example of a Daemon Thread (Non-Java Scenario)

Daemon threads work **in the background**, supporting the main (user) task without interrupting it.

Example: Housekeeping Staff in a Hotel 🏨

- In a hotel, guests (user threads) stay in rooms and use hotel services.
- The **housekeeping staff (daemon thread)** works **in the background**, cleaning rooms, replacing towels, and maintaining the hotel.
- **If all guests leave (all user threads finish), the hotel shuts down, and housekeeping stops working automatically.**

For ex : main method is executed and suddenly its memory becomes full then main thread needs the memory, then you all have to remember that in the background of main thread there is another thread names Garbage Collector.

➤ What it will do?

1. So if the main method memory becoming full then main() needs some memory , so in this case garbage collector delete the unused object from the memory
2. So garbage collector is a thread which is running behind the background and provide some services like resolve the main() problem like memory issues.

CASES :

1. We have to create daemon thread before starting the thread.
2. If we create daemon thread after starting it then it will throw a runtime exception.
3. We can't create a main thread as a daemon thread.
4. Its life depend on the main thread.
5. Most of time daemon thread has low priority , but we can change its priorities.

CODE:

```
package multithreading;
class MyDaemonThread extends Thread {
    public void run() {
        while (true) {
            System.out.println("Daemon thread running...");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

```

public class DaemonThreadExample {
    public static void main(String[] args) {
        MyDaemonThread daemonThread = new MyDaemonThread();
        daemonThread.setDaemon(true); // Setting thread as daemon
        daemonThread.start();

        // Main thread runs for a short time and then terminates
        try {
            Thread.sleep(3000);
            System.out.println("Main thread finished execution.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        // Daemon thread will terminate as soon as main thread finishes
    }
}

```